

UCS503P - Software Engineering Project

EduOS: A Linux-Based Academic Computing Platform

Prototype Report

Submitted by:

1024170276 Dhawal Singh Jamwal

1024170275 Aaditya Kumar Yadav

1024170283 Harshit

BE Computer Science and Engineering

Submitted to:

Dr. Nisha Thakur

Prototype Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	3
1.1	Background and Context	3
1.1.1	Problem Statement	3
1.2	Project Objectives	4
2	Methodology and Design	5
2.1	System Architecture	5
2.1.1	High-Level Design	6
2.1.2	Technical Stack	6
2.2	Implementation Details	7
2.2.1	Module 1: Arch Linux ISO Setup	7
2.2.2	Module 2: User Base and Student Environment Setup	9
2.3	System Diagrams	11
2.3.1	Use Case Diagram	11
2.3.2	Activity Diagram	12
2.3.3	Data Flow Diagrams	13
2.3.4	Sequence Diagram	16
2.3.5	Collaboration Diagram	17
2.3.6	Gantt Chart	18

Introduction

1.1 Background and Context

Modern university computer laboratories depend on a large collection of software tools including compilers, development environments, database systems, networking utilities, operating system tools, and course-specific applications.

Although laboratory computers may use similar hardware, their software environments frequently become inconsistent. Packages may be missing, configuration files may be modified, software versions may differ between machines, and previous users may leave the system in an unexpected state.

These inconsistencies create difficulties for both students and laboratory administrators. Students may encounter situations in which a program works on one machine but fails on another due to differences in installed software or configuration. Administrators must repeatedly install packages, correct configurations, and restore machines to the expected state.

EduOS proposes a Linux-based academic computing platform in which laboratory computers are treated as managed instances of a common academic environment rather than independent general-purpose systems.

The long-term vision of EduOS is to combine operating-system customization, centralized configuration management, academic software distribution, and automated laboratory assessment into one integrated platform.

The present prototype focuses on establishing the basic student-facing operating-system environment and demonstrating multiple independent student identities within the operating system.

1.1.1 Problem Statement

Traditional university laboratory environments are generally configured and maintained on a machine-by-machine basis. This approach introduces several problems:

- Required development tools may be missing from some laboratory machines.
- Compiler, interpreter, library, or application versions may differ between systems.
- Students may unintentionally modify a machine and affect subsequent users.
- Students and instructors may execute programs in different software environments.

- Laboratory administrators must repeatedly perform manual installation and recovery procedures.
- Student laboratory work is often evaluated separately from the environment in which it was produced.

EduOS addresses these problems by proposing a reproducible and centrally manageable Linux environment designed specifically for academic laboratories.

The current prototype demonstrates the first layer of this system: multiple authenticated student operating-system accounts and a personalized academic interface that automatically identifies the logged-in student.

1.2 Project Objectives

The major objectives of EduOS are:

1. **Develop an academic Linux environment:** Create a customized Arch Linux based operating-system image containing the tools and services required for academic laboratory usage.
2. **Provide separate student identities:** Support multiple student accounts with independent Linux user sessions, home directories, and authentication.
3. **Provide a personalized student environment:** Automatically identify the currently authenticated student and display student-specific information and academic tasks.
4. **Support reproducible laboratory environments:** Define software and configuration requirements in a structured form so that similar laboratory systems can be reproduced.
5. **Enable centralized machine management:** Develop a management architecture through which machines can eventually receive laboratory profiles and desired-state configuration.
6. **Support automated laboratory assessment:** Extend the platform to allow students to submit laboratory work that can be executed and evaluated within controlled isolated environments.

Methodology and Design

2.1 System Architecture

EduOS is designed as a layered academic computing platform rather than only as a customized Linux distribution.

The proposed system consists of four primary layers:

1. **EduOS Client Layer**

The operating system installed on laboratory computers. It provides the student desktop environment, development tools, student identity, and the local EduOS software interface.

2. **Management Layer**

Responsible for authentication, machine registration, laboratory profiles, user management, and desired-state configuration.

3. **EduLab Layer**

Responsible for laboratory tasks, student submissions, evaluation requests, and result generation.

4. **Infrastructure Layer**

Provides persistent storage, containerized execution, virtualization, and system deployment infrastructure.

The complete architecture is intended to support the following high-level communication flows:

EduOS Client ↔ Management System ↔ Data Storage

and

Student → Task and Submission Service → Evaluation Worker → Result

2.1.1 High-Level Design

The EduOS architecture separates the user-facing operating system from management and evaluation services.

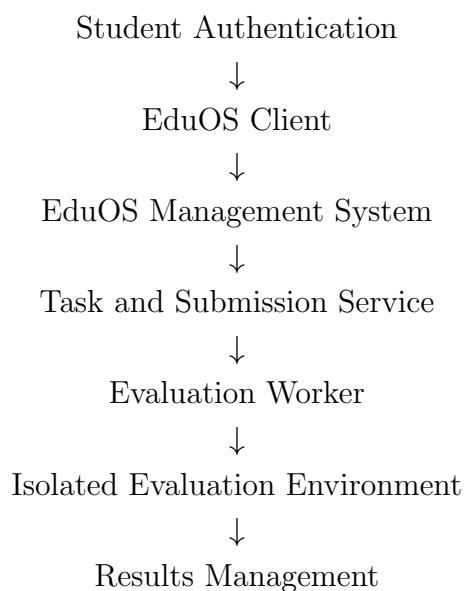
A student first authenticates using an operating-system account. Each student receives a separate Linux identity and home directory.

After authentication, the XFCE desktop session is started. A student-specific EduOS interface is automatically launched. The application determines the current operating-system user and maps the account to academic information such as roll number, student name, and assigned tasks.

For the current proof of concept, this information is stored locally using structured JSON data.

The future architecture replaces this local data source with the EduOS Management System and centralized database.

The planned workflow is therefore:



This separation allows each subsystem to evolve independently. For example, additional laboratory evaluators can be introduced without redesigning the EduOS client operating system.

2.1.2 Technical Stack

The following technologies are used or planned for EduOS:

Technology	Purpose
Arch Linux	Base operating system used for the current EduOS prototype.
ArchISO	Creation of bootable and reproducible EduOS ISO images.
XFCE	Lightweight desktop environment used by EduOS clients.
SDDM	Graphical display manager and Linux user authentication interface.
Python	Implementation of the current EduOS student application and future system services.
Tkinter	Graphical interface used by the current student portal prototype.
JSON	Temporary structured storage for student-specific academic information and tasks.
systemd	Startup service management and EduOS initialization.
QEMU/KVM	Virtualized testing of generated EduOS operating-system images.
Git	Version control for operating-system configuration and project source code.
FastAPI	Planned API framework for the EduOS management and assessment services.
PostgreSQL	Planned persistent storage for centralized users, machines, profiles, tasks, submissions, and results.
Podman	Planned isolation environment for automated execution and evaluation of student programs.

Table 2.1: EduOS Technology Stack

2.2 Implementation Details

2.2.1 Module 1: Arch Linux ISO Setup

The first module focuses on creating the base EduOS operating system environment using Arch Linux and ArchISO.

The purpose of this module is to produce a bootable custom Linux image that contains the graphical environment and system components required by EduOS.

The current prototype uses Arch Linux as the base distribution because ArchISO provides a convenient way to define packages, configuration files, services, and system behavior inside a reproducible bootable image.

The ArchISO profile is maintained under:

`code/archiso/profile`

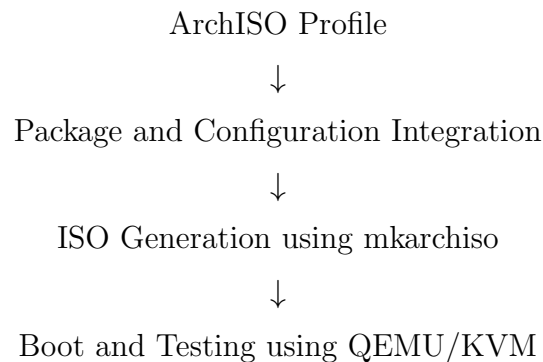
The main components included in the current image are:

- Arch Linux base system
- XFCE desktop environment
- SDDM display manager
- Xorg graphical session
- Mesa graphics support
- Python runtime
- Tkinter for the EduOS student application
- systemd for boot-time service management
- Network and basic administration utilities

The ISO is generated using the `mkarchiso` utility. The build process takes the EduOS ArchISO profile and generates a bootable ISO image that can be tested inside a virtual machine.

The prototype is currently tested using QEMU/KVM. This provides an isolated environment in which the complete operating system can be booted and tested without modifying the host computer.

The overall workflow of this module is:



One major issue encountered during implementation was the graphical session startup. Although user authentication was successful, the XFCE session initially failed because SDDM attempted to launch the XFCE Wayland session.

The problem was solved by configuring SDDM to explicitly use X11. The relevant configuration was placed under:

`/etc/sddm.conf.d/10-eduos-x11.conf`

The resulting configuration ensures that EduOS boots into a stable SDDM and XFCE X11 environment.

At the current stage, this module provides the foundation on which the remaining EduOS features are executed.

2.2.2 Module 2: User Base and Student Environment Setup

The second module focuses on creating and managing multiple student users inside EduOS and providing each student with an independent and personalized academic environment.

Instead of representing students only as records inside the graphical application, the prototype creates actual Linux user accounts.

The following student accounts are currently included in the prototype:

- `student1000`
- `student1001`
- `student1002`
- `student1003`
- `student1004`

A separate administrative account named `eduos` is also retained for system maintenance and debugging.

Each student account has:

- A separate Linux username
- A unique user identity
- An independent home directory
- A separate login session
- A configured password
- An associated student roll number
- Student-specific academic data

The user accounts are provisioned automatically using the EduOS setup script:

`/usr/local/bin/eduos-setup`

This script is executed through a systemd service during system startup.

The service ensures that all required student accounts are created before the SDDM graphical login screen becomes available.

The provisioning mechanism is designed to be idempotent, meaning that running the setup process multiple times maintains the required system state without repeatedly creating duplicate accounts.

After authentication, each student receives a separate XFCE session.

The EduOS student application is implemented using Python and Tkinter and is located at:

```
/usr/local/bin/eduos-student-app
```

The application automatically identifies the currently authenticated Linux user instead of asking the student to perform a second login.

The identified username is used to load the corresponding student information from:

```
/usr/share/eduos/student-data.json
```

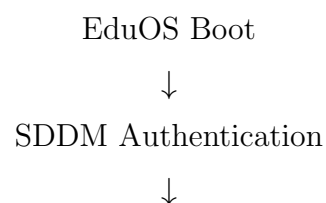
The student interface currently displays:

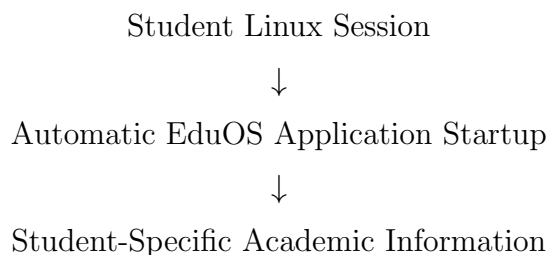
- Student name
- Roll number
- Linux username
- Assigned academic tasks
- Task status
- Due information

Different users are assigned different sample tasks so that the prototype can demonstrate student-specific personalization.

The student application is configured to launch automatically after login and opens in fullscreen mode.

The current user workflow is:





The administrative `eduos` account does not enter the student interface automatically and instead receives access to the normal XFCE desktop. This allows the administrator to inspect users, files, system services, and configuration.

At the current prototype stage, student academic information is stored locally using JSON. In the future, this local data source can be replaced with the centralized EduOS management API and database without changing the fundamental Linux authentication workflow.

2.3 System Diagrams

The following diagrams describe the functional, behavioural, interaction, and data-flow views of the proposed EduOS system.

2.3.1 Use Case Diagram

The use case diagram represents the interactions between the primary EduOS actors and the major services provided by the system.

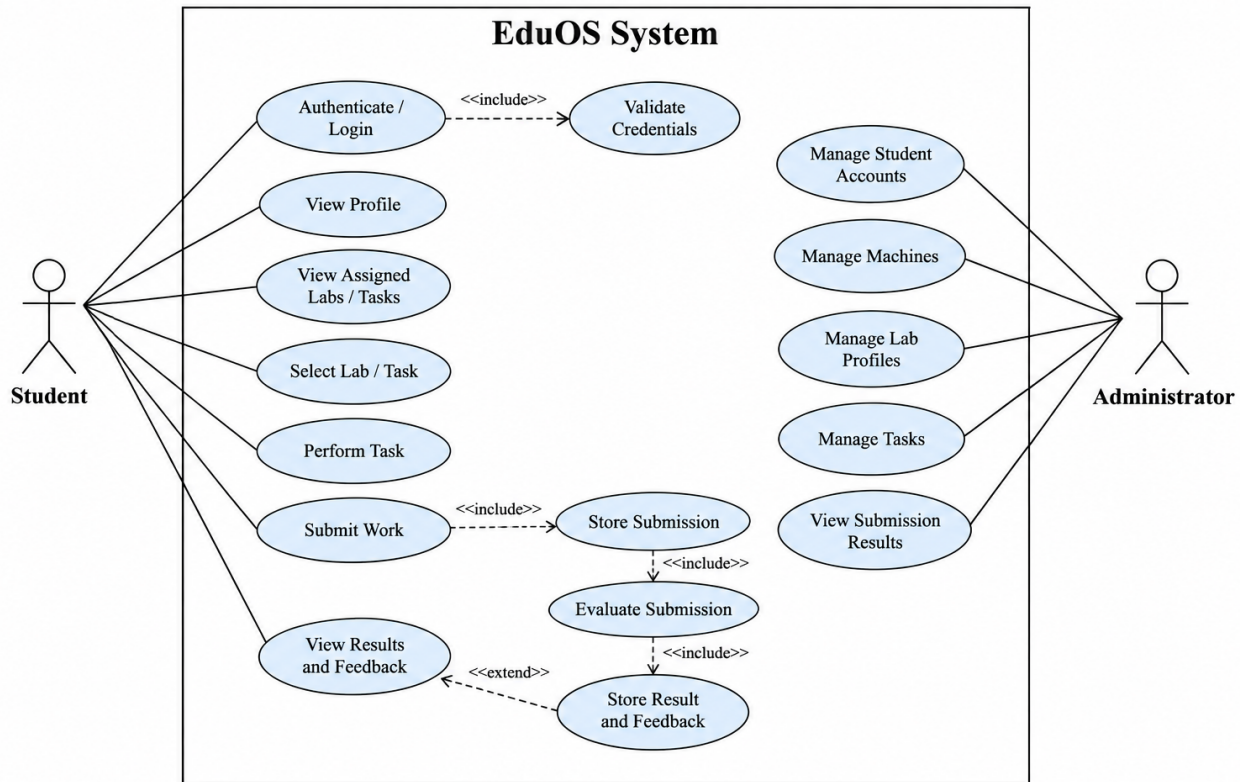


Figure 2.1: EduOS Use Case Diagram

2.3.2 Activity Diagram

The activity diagram represents the workflow of the EduOS system using swimlanes to separate the responsibilities of the student and the major EduOS subsystems.

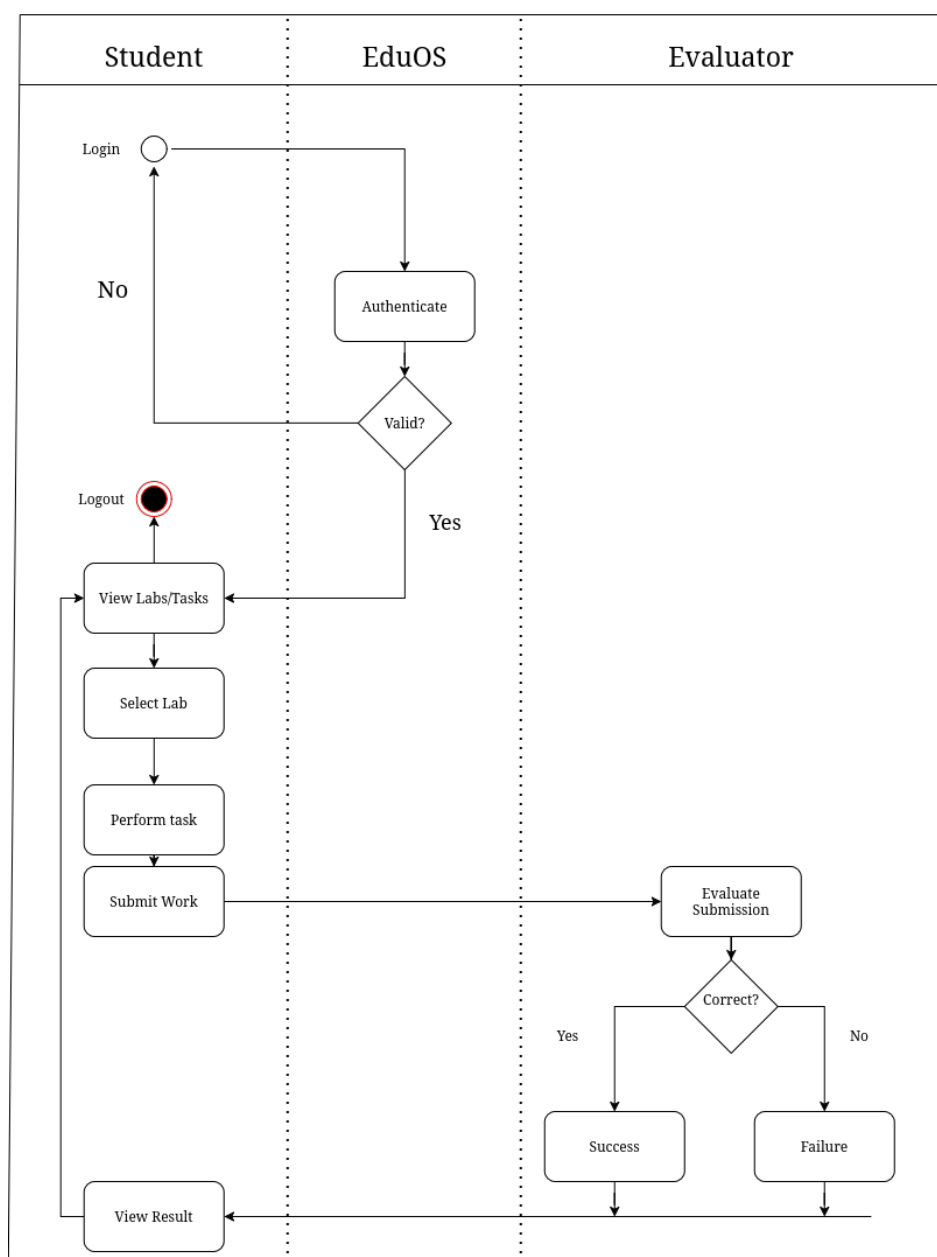


Figure 2.2: EduOS Activity Diagram

2.3.3 Data Flow Diagrams

The data flow diagrams represent how information moves between students, administrators, EduOS processes, evaluation components, and persistent data stores.

Level 0 Data Flow Diagram

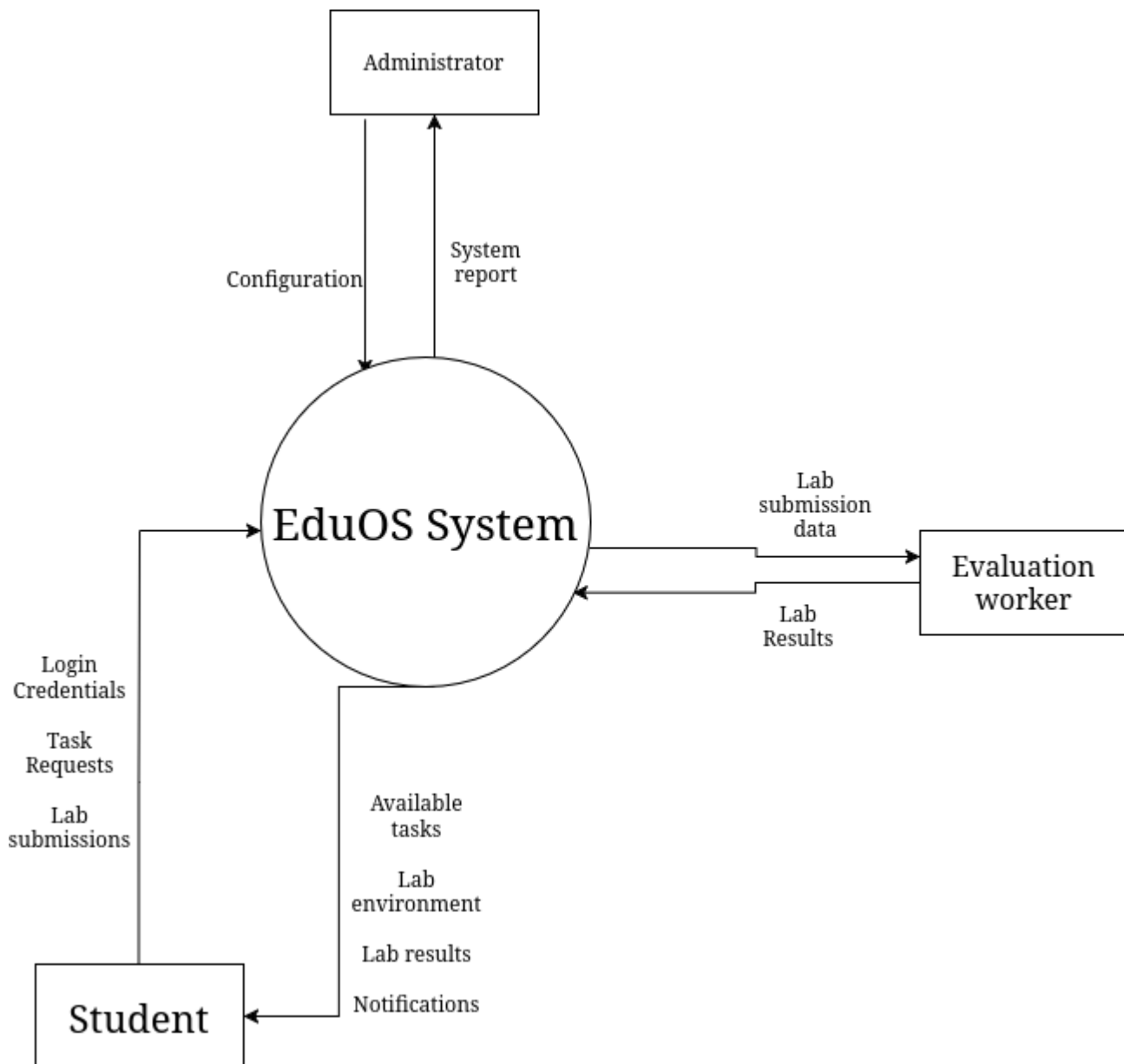


Figure 2.3: EduOS Level 0 Data Flow Diagram

Level 1 Data Flow Diagram

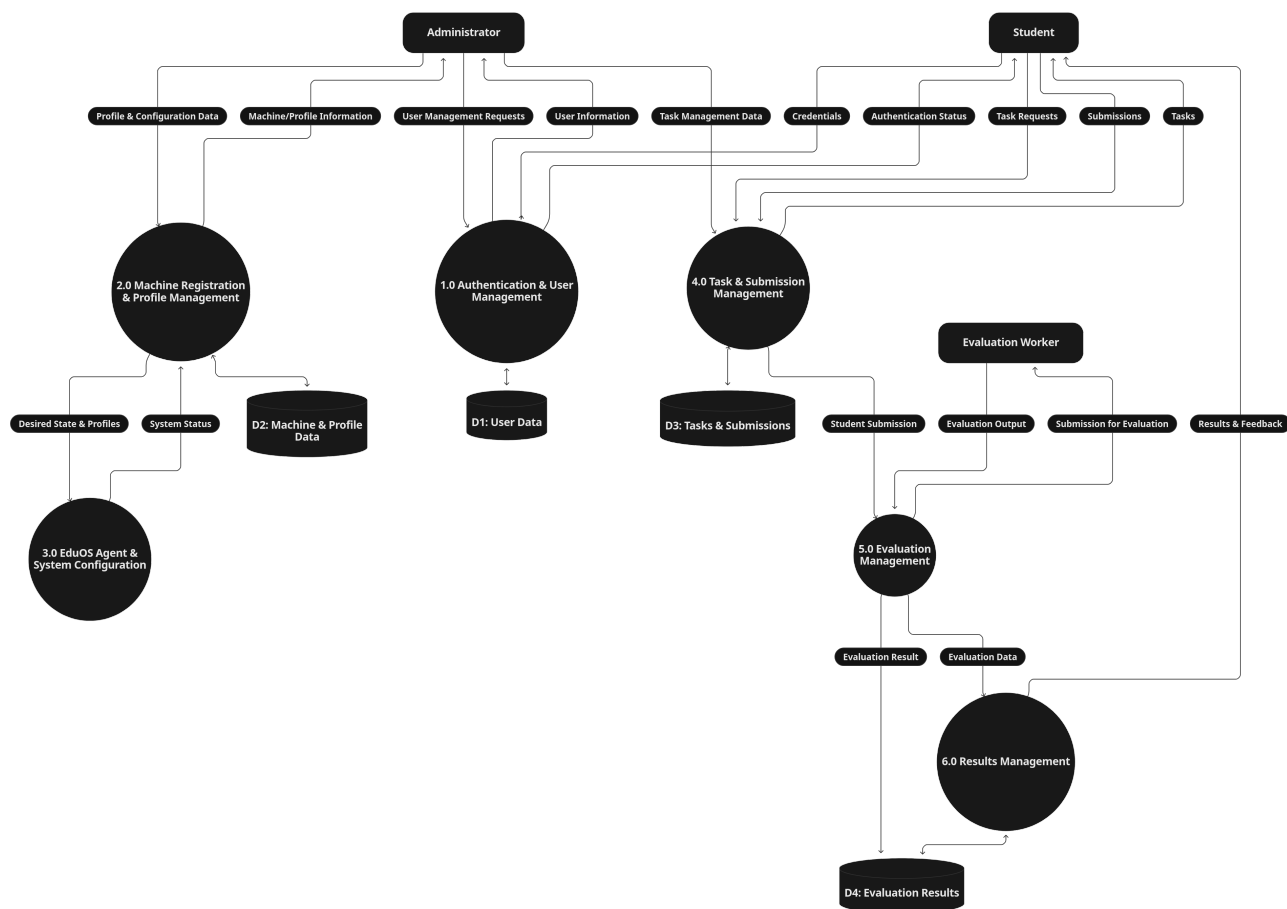


Figure 2.4: EduOS Level 1 Data Flow Diagram

Level 2 Data Flow Diagram

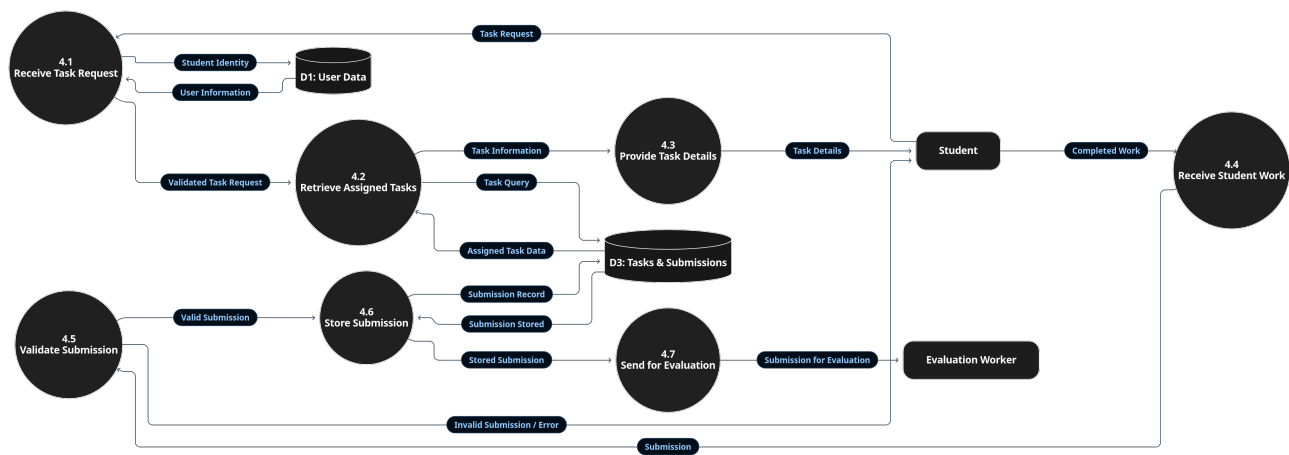


Figure 2.5: EduOS Level 2 Data Flow Diagram

2.3.4 Sequence Diagram

The sequence diagram illustrates the time-ordered interaction between the student, EduOS client, management system, submission service, database, evaluator, and results system.

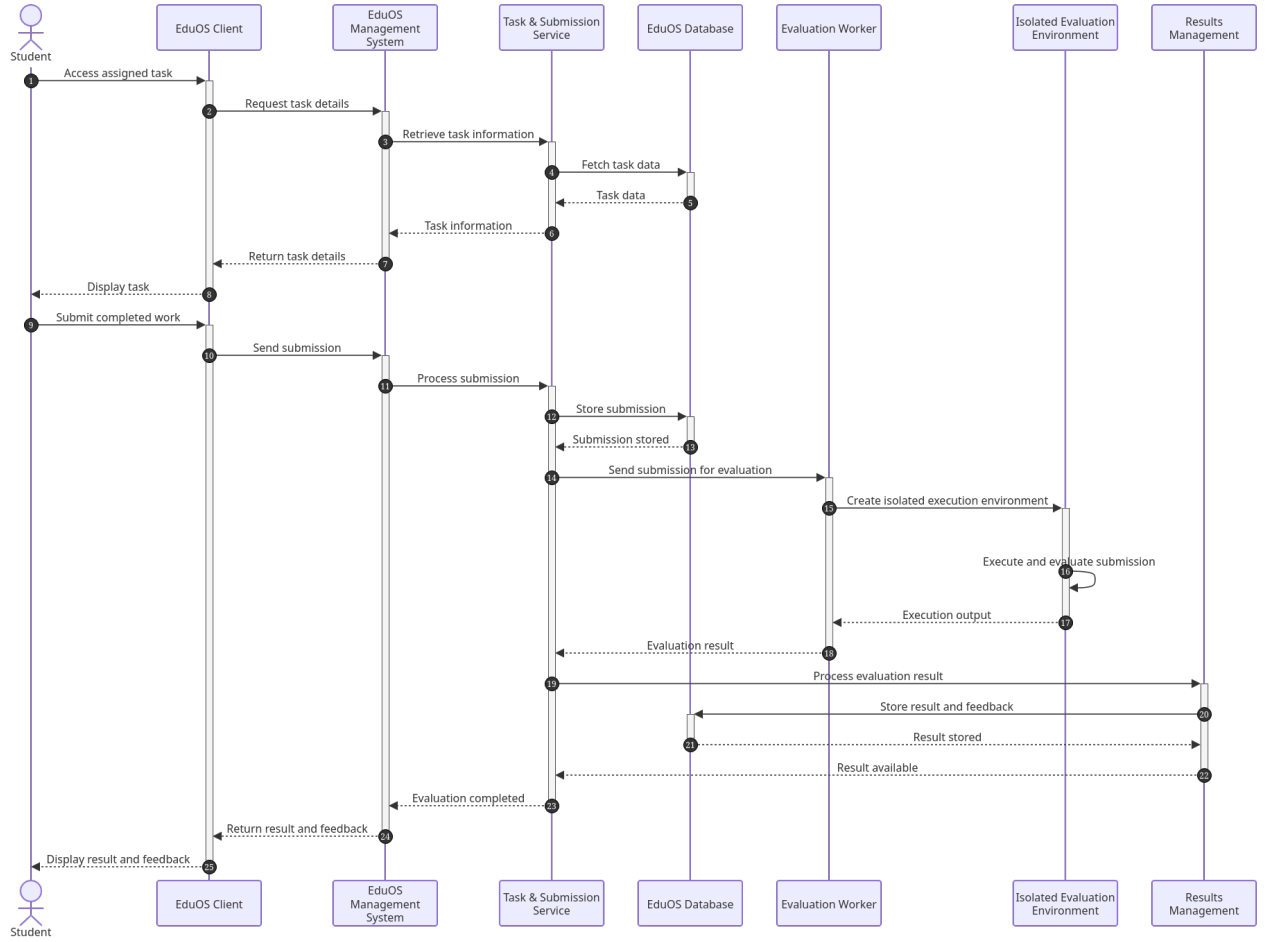


Figure 2.6: EduOS Sequence Diagram

2.3.5 Collaboration Diagram

The collaboration diagram presents the structural interaction between the major EduOS components and the messages exchanged between them.

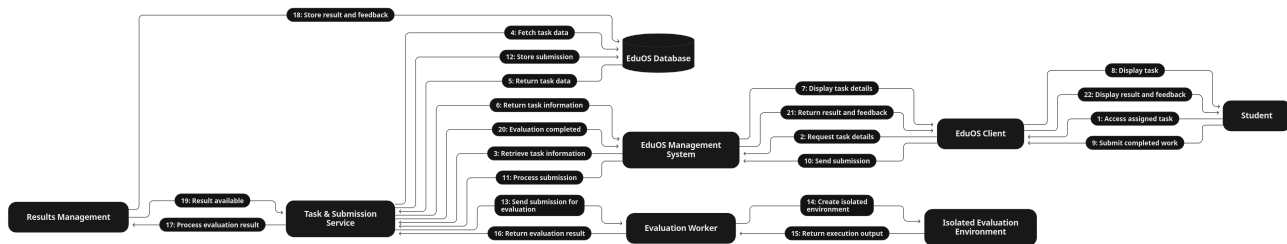


Figure 2.7: EduOS Collaboration Diagram

2.3.6 Gantt Chart

The Gantt chart represents the planned project schedule and the development sequence of the major EduOS modules.

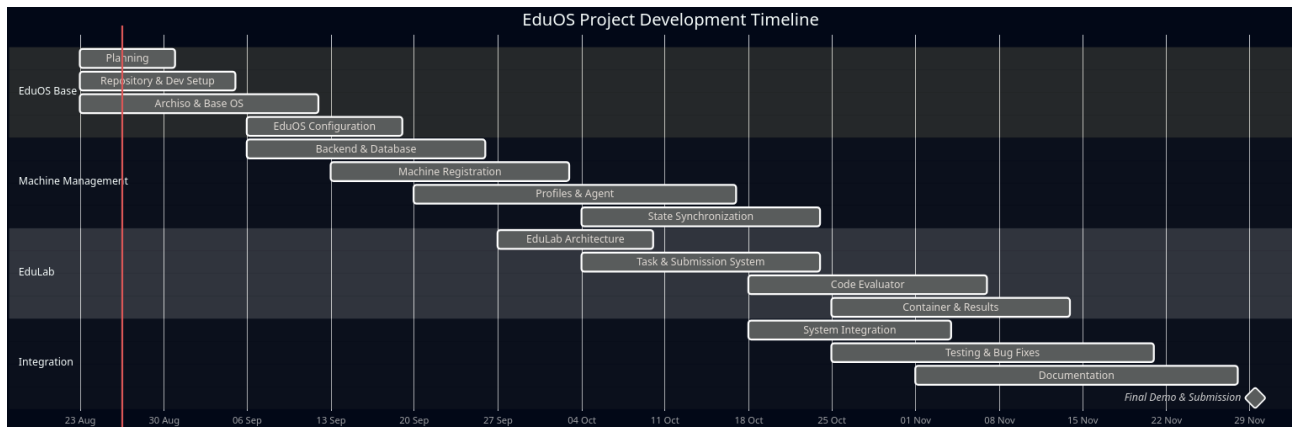


Figure 2.8: EduOS Project Gantt Chart